نسخه‌ی B · README

🔗 چالش «بساز و منتشر کن» — نسخه‌ی B: SnapLink

دوره‌ها: Go — From Beginner to Pro (جلسات ۱ تا ۲۰) + DevOps Bootcamp (جلسات ۱ تا ۳۲) ← Linux ← Go

زمان: ۳ ساعت · **امتیاز:** ۸۰ ساخت + ۲۰ دفاع حضوری + ۱۵ امتیاز اضافه · **نمره‌ی قبولی:** ۶۰

شیوه: انفرادی، از پوشه‌ی خالی · **ابزار:** همه‌چیز آزاد است، حتی هوش مصنوعی

🎯 مأموریت

یک تیم بازاریابی به یک **کوتاه‌کننده‌ی لینک** نیاز دارد: لینک‌های بلند را کوتاه می‌کند، کاربر را به مقصد می‌فرستد و **تعداد کلیک‌ها را دقیق می‌شمارد**؛ حتی وقتی صدها نفر در یک لحظه روی یک لینک کلیک می‌کنند.

تو باید **SnapLink** را با Go بسازی و بعد مثل یک مهندس DevOps **منتشرش کنی**:

- یک میکروسرویس REST با Go که داده‌هایش را در **Redis** نگه می‌دارد
- تست‌های واحد، تست هم‌زمانی و تست HTTP
- اسکریپت‌های Bash برای بررسی محیط و تست دود (smoke test)
- یک ایمیج Docker کوچک و امن
- اجرای کامل با Docker Compose
- یک پایپ‌لاین CI با GitHub Actions
- استقرار روی Kubernetes با **دو رپلیکا**

از یک پوشه‌ی خالی شروع می‌کنی و بعد از ۳ ساعت یک ریپازیتوری Git تحویل می‌دهی.

بعد از آزمون:

۱. مدرس یک **تصحیح‌گر خودکار (auto-grader)** را روی برنامه‌ی در حال اجرای تو و ریپازیتوری‌ات اجرا می‌کند. این برنامه همه‌ی endpointها را صدا می‌زند، لینک‌های خراب و خطرناک می‌فرستد و **صدها کلیک هم‌زمان** شلیک می‌کند تا ببیند شمارش کلیک‌ها دقیق است یا نه.

۲. یک **دفاع حضوری ۷ تا ۱۲ دقیقه‌ای** بدون هوش مصنوعی انجام می‌دهی.

> 📖 قبل از شروع ساعت آزمون، ۱۰ دقیقه وقت مطالعه داری (بدون نوشتن کد): همین فایل و PLAN.md را کامل بخوان. بعد قدم‌به‌قدم طبق **PLAN.md** جلو برو.

🗺️ این چالش چه چیزی را می‌سنجد

مهارت	دوره‌ی Go	بوت‌کمپ DevOps	مرحله
ساختار پروژه، struct، متد، اشاره‌گر	۱ تا ۹	۷ تا ۱۱	S1
رشته‌ها، rune و اعتبارسنجی ورودی	۷	۸ و ۱۱	S1

مهارت	دوره‌ی Go	بوت‌کمپ DevOps	مرحله
اینترفیس و خطاها (`errors.Is`)	۱۰ و ۱۱	۱۲	S1
هم‌روندی (`sync.Mutex`، goroutine، `context`)	۱۲ و ۱۳	۱۳	S1 و S2
JSON و سرور HTTP، ریدایرکت	۱۵ و ۱۷	۱۴	S1
ذخیره‌سازی با Redis و دستورهای اتمیک	۱۸	۱۵، ۲۳ و ۲۴	S1
پیکربندی، middleware، خاموشی مرتب	۱۹ و ۲۰	۱۶ و ۲۰	S1
تست‌نویسی	۱۶	۱۴	S2
لینوکس و اسکریپت Bash	–	۱ تا ۳	S3
Git، GitHub و CI	–	۴ تا ۶	S0، S6 و S8
Docker و Docker Compose	۲۰	۲۱ تا ۲۴	S4 و S5
Kubernetes	–	۲۵ تا ۲۹	S7
مانیتورینگ، Ingress و Helm	–	۲۸ و ۲۹	امتیاز اضافه

📜 قوانین آزمون

۱. **انفرادی است.** کمک گرفتن از هر انسانی (هم‌کلاسی، دوست، فروم، گروه تلگرامی) ممنوع است.

۲. **هر ابزاری آزاد است:** مستندات، ریپازیتوری دوره‌ها، موتور جستجو و دستیارهای هوش مصنوعی (ChatGPT، Claude، Gemini، Copilot و ...).

۳. **مسئول تک‌تک خط‌ها خودت هستی.** در دفاع باید بتوانی هر بخشی از ریپازیتوری‌ات را توضیح بدهی. طبق قانون **«توضیح بده یا از دست بده»**، اگر بعد از دو سؤال ساده‌تر نتوانی یک مورد امتیازی (یک چک‌باکس در PLAN.md) را توضیح بدهی، امتیاز همان مورد صفر می‌شود؛ چه امتیاز ساخت باشد و چه امتیاز اضافه. جمله‌ی «هوش مصنوعی نوشت» توضیح حساب نمی‌شود.

۴. در README خودت یک بخش **«استفاده از هوش مصنوعی» (AI usage)** بنویس: از چه ابزاری، برای چه کاری، و دست‌کم یک جا که هوش مصنوعی اشتباه کرد و تو درستش کردی. استفاده‌ی صادقانه هیچ اشکالی ندارد؛ پنهان‌کاری دارد: بخش AI usage خالی یا غیرواقعی یعنی از دست دادن ۱ امتیاز S8.

۵. سر هر 🚩 **نقطه‌ی بازرسی** در PLAN.md کامیت کن. هر مرحله را روی **شاخه‌ی (branch) خودش** بساز و بعد در `main` ادغام (merge) کن. تاریخچه‌ی Git نمره دارد و خط زمانی کارت را نشان می‌دهد. یک کامیت غول‌آسا در دقیقه‌ی آخر یعنی از دست دادن امتیاز Git و یک دفاع طولانی‌تر.

۶. **توقف قطعی در ۳:۰۰، یعنی دقیقاً سه ساعت بعد از شروع آزمون.** همه‌ی زمان‌های این اسناد (مثل ۰:۱۵ و ۳:۰۰) از لحظه‌ی شروع آزمون حساب می‌شوند و ساعت شروع روی تخته نوشته می‌شود. فقط کامیت‌هایی حساب می‌شوند که قبل از پایان زمان روی GitHub **push** شده باشند.

۷. **قواعد فنی:**

- از **Go نسخه‌ی ۱.۲۶ یا جدیدتر** استفاده کن. الگوهای مسیریابی مثل `GET /{code}` از Go 1.22 آمده‌اند، ولی نسخه‌های جدید کتابخانه‌های Redis و Prometheus به Go 1.26 نیاز دارند. اگر مجبوری با Go قدیمی‌تر کار کنی، نسخه‌های قدیمی‌تر

کتابخانه‌ها را بگیر (این‌ها با Go 1.22 کار می‌کنند): `go get github.com/redis/go-redis/v9@v9.18.0` `github.com/prometheus/client_golang@v1.22.0`

- لایه‌ی HTTP فقط با کتابخانه‌ی استاندارد `net/http` نوشته شود. فریم‌ورک وب (gin، echo، fiber، chi) ممنوع است.
- ماژول‌های مجاز: کلاینت Redis (پیشنهاد: `github.com/redis/go-redis/v9`) و فقط برای امتیاز اضافه‌ی B1، `github.com/prometheus/client_golang`. بقیه‌ی کد فقط با کتابخانه‌ی استاندارد.

۸. **آماده‌سازی سیستم** باید از قبل، طبق «چک‌لیست آماده‌سازی» که یک هفته قبل از آزمون برایت فرستاده شد، انجام شده باشد (خلاصه‌اش پایین همین فایل آمده است). مشکل نصب در روز آزمون دلیل وقت اضافه نیست.

📦 چه چیزی تحویل می‌دهی

یک ریپازیتوری GitHub: عمومی، یا خصوصی با دعوت از حساب GitHub مدرس (Settings → Collaborators؛ نام کاربری مدرس روز آزمون روی تخته نوشته می‌شود). آدرس ریپازیتوری را تا ۰:۱۵ به روشی که مدرس اعلام می‌کند تحویل بده. ساختار پیشنهادی:

```
snaplink/
├── cmd/snaplink/main.go        # سیم‌کشی: config، انتخاب store، سرور، خاموشی مرتب
├── internal/
│   ├── link/                   # Link، اعتبارسنجی URL و کد، تولید کد، خطاها (+ تست)
│   ├── store/                  # اینترفیس Store + memory + redis (+ تست)
│   └── api/                    # handlerها، JSON، نگاشت خطا به status، middleware (+ تست)
├── scripts/
│   ├── check-env.sh
│   └── smoke.sh
├── Dockerfile
├── .dockerignore
├── compose.yaml
├── .env.example
├── .github/workflows/ci.yml
├── k8s/                        # namespace، configmap، secret، redis، app
├── PLAN.md                     # همین نقشه‌ی راه، با بخش «تصمیم‌های من» پر شده
└── README.md                   # خودت: معرفی، نحوه‌ی اجرا، بخش AI usage README
```

هر ساختار تمیز دیگری هم قبول است، به شرط این‌که **دست‌کم سه پکیج** جدا داشته باشد (دامنه، ذخیره‌سازی، HTTP).

📐 قرارداد API (به‌صورت خودکار تصحیح می‌شود)

آدرس پایه: `http://localhost:8080` (پورت با متغیر `PORT` عوض می‌شود)

هر پاسخی که handlerهای تو می‌نویسند (حتی پاسخ خطا) باید هدر `Content-Type: application/json` داشته باشد. فقط این‌ها استثنا هستند: ریدایرکت `302`، پاسخ خالی `204`، و (در امتیاز اضافه‌ی B1) `/metrics` که متن Prometheus برمی‌گرداند.

> ⚠️ تصحیح‌گر فقط به همین قرارداد نگاه می‌کند. نام اشتباه یک فیلد (مثلاً `shortUrl` به‌جای `short_url`) یا status code اشتباه (مثلاً `301` به‌جای `302`) یعنی از دست دادن امتیاز همان بررسی.

شکل داده‌ها

لینک (Link):

```
{
  "code": "aB3xK9",
  "url": "https://go.dev/doc",
  "short_url": "http://localhost:8080/aB3xK9",
  "clicks": 0,
  "created_at": "2026-10-20T09:30:00Z"
}
```

- `short_url` برابر است با `BASE_URL` + `/` + `code` .
- `clicks` یک عدد صحیح است.
- `created_at` با فرمت RFC 3339 است (UTC پیشنهاد می‌شود).

خطا:

```
{ "error": "link not found" }
```

قوانین لینک و کد 🔗

- **آدرس (`url`):** با `net/url` تجزیه می‌شود. scheme فقط `http` یا `https` باشد (بدون حساسیت به حروف بزرگ و کوچک)، host خالی نباشد، و طول کل حداکثر ۲۰۴۸ کاراکتر باشد. در غیر این صورت خطای `400` . پس `""` ، `"not a url"` ، `"http://"` ، `"ftp://x.com"` و `"javascript:alert(1)"` همه `400` هستند.
- **کد تولیدی:** دقیقاً ۶ کاراکتر از `[A-Za-z0-9]` ، ساخته‌شده با `crypto/rand` (**نه** `math/rand`). اگر کد تکراری یا یکی از کلمه‌های رزروشده درآمد (مثلاً `readyz` هم ۶ حرف است!)، دوباره بساز.
- **کد دلخواه (فیلد اختیاری `code`):** ۳ تا ۳۲ کاراکتر، فقط از `[A-Za-z0-9_-]` . کلمه‌های رزروشده‌ی `api` ، `healthz` ، `readyz` و `metrics` (بدون حساسیت به حروف بزرگ و کوچک) رد می‌شوند و خطای `400` دارند. اگر کد قبلاً گرفته شده بود، جواب `409` و `{"error":"code already taken"}` است.
- **هر ریدایرکت** دقیقاً **یک واحد** به `clicks` اضافه می‌کند، به‌صورت اتمیک.

Endpointها

#	متد و مسیر	بدنه	پاسخ موفق	خطاها
۱	`GET /healthz`	–	`200` و `{"status":"ok"}` (هرگز به Redis دست نمی‌زند)	–
۲	`GET /readyz`	–	`200` و `{"status":"ready"}` وقتی Ping به Redis موفق است	`503` و `{"status":"not ready"}`

#	متد و مسیر	بدنه	پاسخ موفق	خطاها
۳	`POST /api/links`	`{"url":"https://go.dev"}` یا `{"url":"…","code":"go-docs"}`	`201` + Link (کلیک ۰)	`400` ، `409`
۴	`GET /api/links`	–	`200` + آرایه‌ی Linkها، **جدیدترین اول** (`created_at` نزولی)؛ اگر خالی بود `[]` (هرگز `null`)	–
۵	`GET /api/links/{code}`	–	`200` + Link با تعداد کلیک فعلی	`404`
۶	`DELETE /api/links/{code}`	–	`204` با بدنه‌ی خالی	`404`
۷	`GET /{code}`	–	`302` با هدر `Location: <url>` و افزایش کلیک	`404` و `{"error":"link not found"}`

معنی و ترتیب خطاها

کد	چه زمانی
`400`	JSON خراب یا نوع اشتباه، `url` نامعتبر، `code` نامعتبر یا رزروشده
`404`	لینکی با این کد وجود ندارد
`409`	کد دلخواه قبلاً گرفته شده است

اگر یک درخواست `POST` چند مشکل داشت، اول `400` و بعد `409` بررسی می‌شود.

پیشنهاد: حجم بدنه‌ی درخواست را با `http.MaxBytesReader` به ۱ مگابایت محدود کن (بدنه‌ی بزرگ‌تر باید خطای `4xx` بگیرد، مثلاً `413` یا `400` ، نه `500` ؛ این مورد نمره ندارد). در Go 1.22 به بعد، الگوی دقیق‌تر برنده است؛ پس `GET /healthz` بر `GET /{code}` اولویت دارد. ⚠️ ولی این فقط وقتی کار می‌کند که هر دو الگو **متد** داشته باشند. اگر مسیری را بدون متد ثبت کنی (مثلاً `mux.Handle("/metrics", …)`)، با `GET /{code}` تداخل (conflict) دارد و برنامه همان لحظه‌ی شروع panic می‌کند؛ پس همیشه با متد ثبت کن (`GET /metrics`). حتماً تستش کن.

یک نمونه‌ی کامل

```
curl -s -X POST localhost:8080/api/links -d '{"url":"https://go.dev/doc"}'
# 201
{"code":"aB3xK9","url":"https://go.dev/doc","short_url":"http://localhost:8080/aB3xK9","clicks":0,
"created_at":"..."}

curl -s -X POST localhost:8080/api/links -d '{"url":"https://kubernetes.io","code":"k8s"}'
# 201 {"code":"k8s",...}

curl -s -X POST localhost:8080/api/links -d '{"url":"https://example.com","code":"k8s"}'
# 409 {"error":"code already taken"}

curl -s -i localhost:8080/aB3xK9
# HTTP/1.1 302 Found
# Location: https://go.dev/doc

curl -s localhost:8080/api/links/aB3xK9
# 200 {"code":"aB3xK9",...,"clicks":1,...}

curl -s -X POST localhost:8080/api/links -d '{"url":"javascript:alert(1)"}'
# 400 {"error":"invalid url"}

curl -s -i -X DELETE localhost:8080/api/links/k8s
# HTTP/1.1 204 No Content
```

پیکربندی با متغیرهای محیطی

متغیر	پیش‌فرض	معنی
`PORT`	`8080`	پورت HTTP
`STORE`	`redis`	نوع ذخیره‌سازی: `redis` یا `memory`
`REDIS_ADDR`	`localhost:6379`	آدرس Redis
`REDIS_PASSWORD`	خالی	رمز Redis (در Kubernetes از Secret خوانده می‌شود)
`BASE_URL`	`http://localhost:8080`	برای ساختن `short_url` (بدون `/` در انتها)

- با `STORE=memory` برنامه **بدون Redis** و با یک store درون‌حافظه‌ای اجرا می‌شود. این حالت برای تست‌ها و شروع سریع کار است.
- اگر پیکربندی نامعتبر بود (مثلاً `STORE` مقدار ناشناخته داشت)، برنامه باید با یک پیام واضح در لاگ **فوراً خارج شود** (fail fast).
- با سیگنال `SIGINT` یا `SIGTERM` برنامه **مرتب خاموش می‌شود**: درخواست‌های در حال اجرا را تمام می‌کند (حداکثر ۱۰ ثانیه)، اتصال Redis را می‌بندد و پیامی در لاگ می‌نویسد. Kubernetes هم برای متوقف کردن pod همین `SIGTERM` را می‌فرستد.
- لاگ‌ها با `log/slog` به‌صورت JSON چاپ شوند: برای هر درخواست یک خط با `method`، `path`، `status` و `duration_ms`.

اینترفیس Store (راهنمایی)

می‌توانی از همین اینترفیس استفاده کنی. دو پیاده‌سازی لازم است: یکی درون‌حافظه‌ای با `sync.Mutex` و یکی Redis.

```
type Store interface {
    Create(ctx context.Context, l link.Link) (link.Link, error) // اگر کد وجود داشت: ErrCodeTaken
    Get(ctx context.Context, code string) (link.Link, error)    // ErrNotFound
    List(ctx context.Context) ([]link.Link, error)              // جدیدترین اول
    Delete(ctx context.Context, code string) error              // ErrNotFound
    Hit(ctx context.Context, code string) (string, error)       // کلیک++ به‌صورت اتمیک و برگرداندن url
    Ping(ctx context.Context) error
    Close()                                                     // برای خاموشی مرتب
}
```

خطاهای ثابت (sentinel) پیشنهادی در پکیج `link`: `ErrInvalidURL`، `ErrInvalidCode`، `ErrCodeTaken`، `ErrNotFound`.

طراحی کلیدهای Redis (پیشنهادی، می‌توانی طراحی خودت را داشته باشی):

کلید	نوع	محتوا
`link:{code}`	HASH	`url`، `clicks`، `created_at`
`links`	ZSET	عضو = کد، امتیاز = زمان ساخت (میلی‌ثانیه) برای لیست «جدیدترین اول»

🏆 امتیازبندی

مرحله	موضوع	امتیاز
S0	شروع: «تصمیم‌های من» (روش Q)، ریپازیتوری، جریان شاخه‌ها	۵
S1	برنامه‌ی Go: دامنه، API، Redis، هم‌روندی	۲۵
S2	تست‌ها: جدول‌محور، هم‌زمانی، HTTP	۸
S3	لینوکس و اسکریپت‌های Bash	۷
S4	Docker	۹
S5	Docker Compose	۷
S6	GitHub Actions با CI	۶
S7	Kubernetes	۱۰
S8	مستندات، یادداشت AI و تاریخچه‌ی Git	۳
	جمع ساخت	**۸۰**
D	دفاع حضوری (بدون هوش مصنوعی)	۲۰
	جمع اصلی	**۱۰۰**
B1 تا B4	امتیاز اضافه: متریک Prometheus، Ingress، Helm، لینک‌های تاریخ‌دار	۱۵+

جزئیات امتیاز تک‌تک کارها در **PLAN.md** آمده است.

رتبه

نمره	رتبه
۹۰ تا ۱۱۵	🏆 **افسانه (Legend)**
۸۰ تا ۸۹	🥇 **حرفه‌ای (Pro)**
۷۰ تا ۷۹	🥈 **سازنده (Builder)**
۶۰ تا ۶۹	🥉 **قبول (Pass)**
زیر ۶۰	🔁 **آزمون مجدد (Retake)**

نمره‌ی نهایی قبل از تعیین رتبه و قبولی به نزدیک‌ترین عدد صحیح گرد می‌شود (مثلاً ۵۹٫۵ می‌شود ۶۰ و ۵۹٫۴ می‌شود ۵۹). امتیاز اضافه هم در رسیدن به نمره‌ی قبولی حساب می‌شود.

🔍 چطور تصحیح می‌شوی

۱. **در ۳:۰۰ دست از کار بکش.** برنامه‌ات را در Kubernetes در حال اجرا نگه دار و با `port-forward` روی `localhost:8080` در دسترس بگذار (اگر Compose هنوز روشن است، اول `docker compose down` بزن تا پورت 8080 آزاد شود). اگر Kubernetes کار نکرد، برنامه را با Docker Compose اجرا کن. بعد از ۳:۰۰ هیچ چیزی را در کد یا استقرار عوض نکن؛ تصحیح روی آخرین کامیت pushشده انجام می‌شود.

۲. در نوبت دفاع، مدرس تصحیح‌گر خودکار را **روی لپ‌تاپ خودت** اجرا می‌کند. این برنامه:

- همه‌ی endpointها را صدا می‌زند و URLهای خراب و خطرناک، کدهای نامعتبر و کدهای تکراری می‌فرستد. **سرور تو هرگز نباید crash کند یا برای ورودی بد `500` برگرداند.**
- **۱۰۰ کلیک هم‌زمان** روی یک لینک و **۲۰ درخواست هم‌زمان** با یک کد دلخواه یکسان می‌فرستد. شمارش باید دقیقاً ۱۰۰ باشد و فقط یکی از آن ۲۰ درخواست موفق شود.
- ریپازیتوری، Dockerfile، فایل workflow، Compose و فایل‌های Kubernetes تو را بررسی می‌کند.

۳. **دفاع حضوری (۲۰ امتیاز، بدون هوش مصنوعی):** توضیح مسیر یک درخواست در کد خودت، یک سؤال «چرا؟» درباره‌ی تصمیم‌هایت، رفع یک خرابی که مدرس در استقرارت ایجاد می‌کند، و یک تغییر کوچک زنده در کمتر از ۳ دقیقه.

۴. **نمره‌ی نهایی** = ساخت (حداکثر ۸۰) + دفاع (حداکثر ۲۰) + امتیاز اضافه (حداکثر ۱۵).

۵. **امتیاز هم‌رونده (۳) فقط روی datastore واقعی حساب می‌شود.** اگر برنامه با `STORE=memory` اجرا شود، این ۳ امتیاز صفر می‌شود؛ پس برنامه‌ای که تصحیح می‌شود باید به Redis وصل باشد.

> 💡 **نکته:** قبل از شروع کدنویسی، چک‌لیست امتیازها را در PLAN.md بخوان. وقتی بدانی امتیازها کجا هستند، جمع کردنشان آسان است.

🧰 یادآوری چک‌لیست آماده‌سازی (باید از قبل انجام شده باشد)

این‌ها را باید طبق «چک‌لیست آماده‌سازی» که یک هفته قبل از آزمون گرفتی، از قبل روی لپ‌تاپ خودت آماده کرده باشی. خلاصه‌اش:

- [ ] **Go** نسخه‌ی ۱.۲۶ یا جدیدتر (`go version`). در Linux و WSL، `go test -race` به کامپایلر C نیاز دارد: `sudo apt install -y build-essential` (در macOS لازم نیست).
- [ ] **Git** و حساب **GitHub** با کلید SSH که کار می‌کند (`ssh -T git@github.com`).
- [ ] **Docker** روشن باشد (`docker run --rm hello-world`).
- [ ] **kubectl** و **kind** (پیشنهادی) یا **minikube**، با کلاستری به نام `exam` که از قبل ساخته‌ای و **پاکش نکرده‌ای**. با `kubectl get nodes` باید وضعیت نود `Ready` باشد. دستور ساخت (فقط یک بار):

```
kind create cluster --name exam
```

- [ ] **curl** و **Python 3** نصب باشند (تصحیح‌گر با Python 3 روی لپ‌تاپ تو اجرا می‌شود). **jq** اختیاری است.
- [ ] ایمیج‌ها دانلود شده باشند. تگ `golang` را با نسخه‌ی Go خودت هماهنگ کن (اگر `go version` می‌گوید `go1.27`، بزن `golang:1.27-alpine`):

```
docker pull golang:1.26-alpine
docker pull redis:7-alpine
docker pull alpine:3.22
```

- [ ] ماژول‌های Go از قبل در cache باشند (این دستورها را **داخل پوشه‌ی پروژه‌ات اجرا نکن**):

```
mkdir -p /tmp/warmup && cd /tmp/warmup && { [ -f go.mod ] || go mod init warmup; }
go get github.com/redis/go-redis/v9 github.com/prometheus/client_golang/prometheus
```

- [ ] **کاربران macOS:** ترمینال پیش‌فرض (zsh) علامت `#` را در خط فرمان کامنت حساب نمی‌کند. یک بار این را بزن و ترمینال را دوباره باز کن: `echo 'setopt interactivecomments' >> ~/.zshrc`
- [ ] **کاربران ویندوز:** همه‌ی کارها داخل WSL2 (Ubuntu) انجام شود و WSL integration در Docker Desktop روشن باشد.
- [ ] **اگر اینترنت محدود است (ایران):**
  - Docker Hub: یک `registry-mirrors` تنظیم کن (در Docker Desktop از Settings → Docker Engine؛ در Engine لینوکس در فایل `/etc/docker/daemon.json` و بعد `sudo systemctl restart docker`) یا از VPN استفاده کن. آدرس mirror را مدرس اعلام می‌کند.
  - Go: `go env -w GOPROXY=https://goproxy.io,direct`. این فقط روی سیستم خودت اثر دارد. اگر داخل `docker build` دانلود ماژول‌ها شکست خورد، در مرحله‌ی ساخت Dockerfile و قبل از `RUN go mod download` این خط را بگذار: `ENV GOPROXY=https://goproxy.io,direct`
  - کلاستر kind از mirror داکر استفاده نمی‌کند. ایمیج Redis را از Docker خودت داخل کلاستر بارگذاری کن: `kind load docker-image redis:7-alpine --name exam` (ایمیج برنامه‌ات را هم بعد از هر build همین‌طور load کن: `kind load docker-image snaplink:v1 --name exam`).
  - از ایمیج‌های `gcr.io` و `registry.k8s.io` (مثل distroless) دوری کن؛ برای مرحله‌ی آخر Dockerfile از `scratch` یا `alpine` استفاده کن.

موفق باشی! یادت باشد: **لینکی که کلیک‌هایش را گم کند، به درد بازاریاب نمی‌خورد.** 🐹